Deadlock

A deadlock occurs when there are threads $T_1, \ldots, T_n$

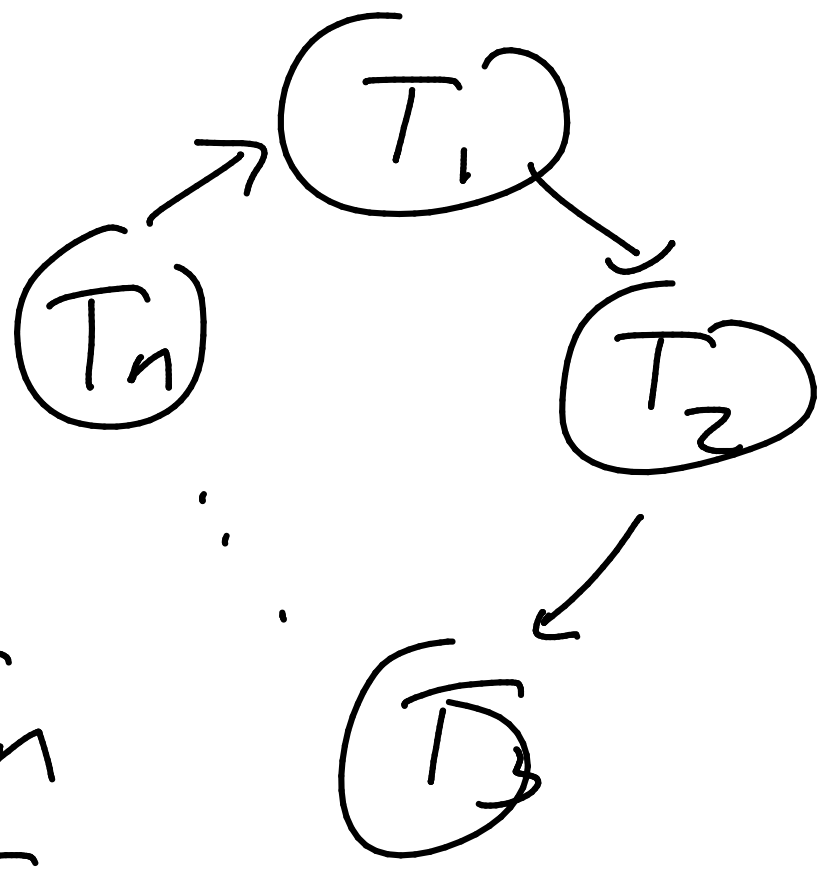
T_1 is waiting on T_2

T_2 is waiting on T_3

$\vdots$

T_{n-1} ---- on T_n

T_n ---- on T_1



Why in BankAccount example

T1	T2
line G (lock on one)	
line H	
line A (lock #2 on one)	
	line G (lock on two)
	line H
	line A (lock #2 on two)
	line B
	line C (release lock #2)
	line I
	line D (blocked trying to lock one)

line B
line C (release lock #2)
line I
line D (blocked trying to lock two)

We fixed by forcing an ordering on the locks (lower id first).

Can we prove this works?

By contradiction: suppose it deadlocks.

The thread 1 must have locked X and is waiting on Y

thread 2 ..... Y

and is waiting on X

We lock the smaller id first.

So id X < id Y can't be

and id Y < id X

read/write locks are nice esp if you do a lot of reading (you get more parallelism that way).

But what could go wrong?

- read locks can be shared
- writelocks cannot

One problem is that the thread that wants a write lock might <u>starve</u> (waits forever for the lock) because read locks keep coming and going (overlapping) and never releasing entirely

Some R/W lock implementations (including Kotlin/Java) have a "fair" option to give write a chance

- e.g. if a write is waiting, allow no more reads to be added

For improving multiple thread algs,
mechanism usually called notify/wait
("conditions")
where wait = release lock and wait
until you're notified
notify = wake up another thread)

In Java, you call wait/notify on
any object

In Kotlin, this is more restricted,
and you have to do it on a Condition
object
- await
- signal